

ХАКАТОН · ЗАДАЧА 2 · КОМАНДА «КОЛОМОТ»

# Умный роутинг выплат

Движок распределения выплат между платёжными провайдерами:  
допуск, выбор, каскад, объяснение и рекомендации

Фёдор Филитович · Артём Корсаков

Ruby 3.3, только стандартная библиотека · 116 тестов · 0 внешних зависимостей

[github.com/T0vich/kolomot-smart-payout-routing](https://github.com/T0vich/kolomot-smart-payout-routing)

## «Выбрать подходящего партнёра» — это три разных вопроса

### Кому **можно**?

Ответ бинарный. Лимиты, банки, реквизиты, маржа. Нарушать нельзя никогда — независимо от того, какая стратегия включена.

### Кого **лучше**?

Ответ — всегда компромисс. Семь целей из ТЗ конфликтуют между собой, и это не баг постановки, а её суть.

### Что если **отказал**?

Нужна последовательность попыток, которая ни на одном шаге не нарушает жёсткие ограничения.

**Пример конфликта прямо из ТЗ.** Цель — 40 % заявок на вірау. Факт — вірау почти исчерпал дневной максимум, а на payflow висит обязательство «не менее 2 млн ₽ в сутки». Обе цели корректны. Одновременно выполнить нельзя. Кому отдаём заявку — и как это объяснить?

Большинство ошибок в таких системах — от попытки решить все три вопроса одним механизмом.

## Два слоя, которые нельзя смешивать

**Hard-constraints** — допуск, применяются **до** выбора стратегии. **Soft-goals** — предпочтение, работают **только внутри** допустимого пула.

```
operation
|
▼ ProviderPool#advance_clock — двигаем модельные часы, освобождаем слоты
|                               завершившихся заявок
▼ Constraints::Chain — 9 правил допуска; отсеб сразу пишется в attempts
|                               ↓ допустимый пул
▼ Strategies × 7 — каждая даёт под-скор [0..1] по всем кандидатам
|
▼ CompositeScorer — взвешенная сумма + разрешение ничьи
|                               ↓ ранжированный список
▼ Router#walk_cascade — попытка → отказ → следующий кандидат
|                               ↓ никто не принял
▼ Router#fall_back — self-provider spaceraumts
|
▼ Simulator + ProviderPool#occupy — исход, задержка, занять слот и лимит
```

Стратегия физически не может «уговорить» роутер нарушить лимит. Альтернативу — единый скоринг со штрафом за нарушение — мы отвергли: там нарушение перестаёт быть невозможным и становится просто дорогим.

# 9 жёстких ограничений — все из ТЗ, по классу на правило

| Правило              | Код причины в attempts                                   | Правило                | Код причины в attempts                  |
|----------------------|----------------------------------------------------------|------------------------|-----------------------------------------|
| Статус провайдера    | provider_inactive                                        | Банковский фильтр      | bank_not_in_list · bank_in_exclude_list |
| Диапазон суммы чека  | amount_below_minimum · amount_exceeds_limit              | Маржа                  | negative_margin                         |
| Дневной максимум     | daily_limit_exceeded                                     | Реквизиты              | no_requisites                           |
| Одновременные заявки | in_progress_count_exceeded · in_progress_amount_exceeded | Интенсивность          | rate_limit_exceeded                     |
|                      |                                                          | Потолок оборота (наше) | turnover_max_reached                    |

Отсев всегда объяснён числами, а не общими словами:

```
{ "provider": "payflow", "decision": "skipped", "stage": "hard_filter",  
  "reason": "amount_exceeds_limit", "details": "150000 > limit_amount_max 50000" }
```

Состав и порядок правил задаётся config/routing.yml — любое выключается без правки кода.

# Семь стратегий распределения — все семь из ТЗ

| # | Стратегия          | Поле                      | Что делает                                                                 |
|---|--------------------|---------------------------|----------------------------------------------------------------------------|
| 1 | Доля по количеству | traffic_percentage        | Минимизирует <b>суммарное</b> отклонение всех фактических долей от целевых |
| 2 | Доля по объёму     | volume_share_pct          | То же самое, но доли считаются в рублях                                    |
| 3 | Очередь в каскаде  | priority                  | Ранг считается <b>среди доступных</b> , а не по всему списку               |
| 4 | По сумме чека      | amount_bands              | Диапазон влияет на <b>предпочтение</b> , а не только отсекает              |
| 5 | По конверсии       | conversion_24h            | Заявленная, смешанная с фактической по истории                             |
| 6 | По интенсивности   | requests_per_minute_limit | Текущая загрузка влияет на <b>выбор</b> , а не только проверяется потолок  |
| 7 | Фин. обязательства | daily_turnover_min / max  | Дотягивает минимальный оборот, тормозит у максимума                        |

Каждая — отдельный класс, каждая нормирует свой сигнал в [0..1], каждая включается весом в профиле. Вес 0 — стратегия не участвует.

# Все семь целей сразу — взвешенный композитный балл

$$\text{score(провайдер)} = \sum ( \text{под-скор}_i \times \text{вес}_i ) / \sum \text{вес}_i$$

Веса профиля **balanced**, сдаваемого по умолчанию:

|                    |                                                                                     |      |                                      |
|--------------------|-------------------------------------------------------------------------------------|------|--------------------------------------|
| Доля по количеству |   | 0.26 | прямое требование ТЗ и договора      |
| Конверсия          |   | 0.18 | слать туда, где не проходит, дорожке |
| Доля по объёму     |   | 0.16 | вместе с долей по количеству — 0.42  |
| Фин. обязательства |   | 0.14 | стратегия и так разрывная            |
| Каскад             |   | 0.10 | уточняющий сигнал                    |
| Диапазон суммы     |   | 0.09 | жёсткая версия уже отработала        |
| Загрузка           |  | 0.07 | жёсткая версия уже отработала        |

Веса живут в **config/routing.yml**, а не в коде: завтра приоритетом станет конверсия — меняется одна строка. Профиль `conversion_first` уже в репозитории.

# Что делаем, когда цели указывают на разных провайдеров

## 1. Разрыв баллов мал

Применяется явный порядок политик **conflict\_order**:

фин. обязательства → доля по количеству → конверсия → доля по объёму → каскад → диапазон суммы → загрузка

## 2. Цели равны полностью

Детерминированный разрыв: сначала **priority**, затем имя провайдера.

Один и тот же вход всегда даёт один и тот же выход — это отдельный тест.

## 3. Цель невыполнима

Требуемая доля относится к провайдеру, которого отсекают hard-constraints.

Цель **не блокирует** роутинг: заявка уходит дальше, расхождение — в отчёт с рекомендацией.

**Почему обязательства выше долей.** За недобор минимального оборота платят деньгами по договору. Целевая доля — ориентир, её можно добрать следующими заявками. Порядок задан конфигом и проверяется при старте: неизвестная стратегия в `conflict_order` — ошибка запуска, а не тихое игнорирование.

# Не «выбрали vipay», а почему он и почему не остальные

ор\_101, 15 000 ₽, Сбербанк. Допустимы все трое — и цели расходятся: по диапазону суммы лучше payflow, по загрузке — quickpay, по долям и конверсии — vipay.

| Слагаемое × вес                                   | vipay       | payflow     | quickpay    |
|---------------------------------------------------|-------------|-------------|-------------|
| Доля по количеству × 0.26                         | 1.00 → 0.26 | 0.67 → 0.17 | 0.00 → 0.00 |
| Конверсия × 0.18                                  | 1.00 → 0.18 | 0.27 → 0.05 | 0.00 → 0.00 |
| Доля по объёму × 0.16                             | 1.00 → 0.16 | 0.25 → 0.04 | 0.00 → 0.00 |
| Фин. обязательства × 0.14                         | 0.18 → 0.03 | 0.50 → 0.07 | 0.50 → 0.07 |
| Каскад × 0.10 · Диапазон × 0.09 · Загрузка × 0.07 | 0.15        | 0.14        | 0.08        |
| Итоговый балл                                     | 0.77        | 0.47        | 0.15        |

В attempts у выбранного — балл, разложение по слагаемым и текст с ближайшим конкурентом. У остальных — конкретная причина: **amount\_exceeds\_limit**, **bank\_not\_in\_list**, **lower\_score** — и расшифровка числами.



# Различаем два события, названные в ТЗ одним словом

| Событие                             | Что значит                                        | Что делает роутер                                                           |
|-------------------------------------|---------------------------------------------------|-----------------------------------------------------------------------------|
| Провайдер <b>не принял</b> заявку   | Ретраибельный отказ, заявка в работу не ушла      | Исключаем из пула → следующий кандидат. Кандидаты кончились → self-provider |
| Провайдер <b>принял и обработал</b> | Терминальный исход: approved / rejected / expired | Маршрут не меняется, исход пишется в simulated_result                       |

Каскад в действии — профиль demo\_failover, заявка op\_103 на 150 000 ₽:

|               |          |                        |                                    |
|---------------|----------|------------------------|------------------------------------|
| vipay         | skipped  | amount_exceeds_limit   | 150000 > limit_amount_max 100000   |
| payflow       | skipped  | amount_exceeds_limit   | 150000 > limit_amount_max 50000    |
| quickpay      | skipped  | provider_declined      | провайдер отказался принять заявку |
| spacepayments | selected | fallback_self_provider | внешних допустимых не осталось     |

В сдаваемом профиле искусственные отказы выключены осознанно: эталон организаторов требует для op\_103 строго quickpay, а он там единственный допустимый — случайный отказ увёл бы заявку в spacepayments и провалил проверку. Механика не исчезла: **rake demo** и 4 теста.

# Лимиты резервируем, а не считаем пост-фактум

## Наивная реализация

`daily_approved_amount` растёт только после подтверждения выплаты.

Десять заявок, отправленных подряд, все проходят проверку дневного лимита — и лимит продан десять раз.

## Наша реализация

Сумма резервируется в момент выбора провайдера:

**занято = `daily_approved_amount` + `daily_reserved_amount`**

При завершении резерв снимается. То же для `in-progress count` и `amount`.

**Побочная выгода.** Валидатор организаторов считает лимиты по исходному снимку `providers.json`. Наша проверка строго жёстче — значит, выбранный нами провайдер гарантированно входит в его список допустимых. Результат: **29 проверок пройдено, 0 ошибок, 0 предупреждений.**

Плюс модельное время: заявка держит слот `latency_sec` и освобождает его. На публичной очереди из 10 заявок разницы не видно, на длинной тестовой — решает, упрётся провайдер в лимит одновременных заявок или нет.

# Отчёт называет причину расхождения и параметр к изменению

| Провайдер | Заявок | Факт   | Цель   | Откл. | Утилизация дневного лимита | Конверсия: снимок → факт |
|-----------|--------|--------|--------|-------|----------------------------|--------------------------|
| viраy     | 4      | 40.0 % | 40.0 % | 0.0   | 65.8 %                     | 0.87 → 0.78              |
| payflow   | 3      | 30.0 % | 35.0 % | −5.0  | 98.3 %                     | 0.91 → <b>0.47</b>       |
| quickpay  | 3      | 30.0 % | 25.0 % | +5.0  | 16.2 %                     | 0.79 → 0.68              |

**Главная находка по данным.** Провайдер с самой высокой заявленной конверсией (payflow, 0.91) на деле худший — 0.47 по 100 историческим операциям, дрейф −43.6 п.п. Роутинг, который берёт conversion\_24h как есть, будет систематически слать заявки туда, где они не проходят. Поэтому у нас конверсия смешанная: history\_blend = 0.3.

Рекомендации называют провайдера, число и параметр:

- payflow выбрал 98.29 % дневного лимита – поднять daily\_amount\_limit до 3 600 000 ₺ или снизить его долю трафика, иначе заявки начнут отсеиваться
- viраy отсеян 4 раза по фильтру банка – расширить banks или подключить недостающие банки, иначе целевая доля недостижима

# Переиграли 100 исторических операций через наш роутер

| Провайдер                     | Цель   | Было в истории | Стало у нас |
|-------------------------------|--------|----------------|-------------|
| payflow                       | 35.0 % | 19.0 %         | 25.0 %      |
| quickpay                      | 25.0 % | 40.0 %         | 38.0 %      |
| vipay                         | 40.0 % | 41.0 %         | 37.0 %      |
| Суммарное отклонение от целей | —      | 32.0 п.п.      | 26.0 п.п.   |

Метрика намеренно не замкнута на себя: ожидаемая конверсия считается по фактической успешности провайдеров из той же истории, а не по нашему симулятору.

**29 из 100**

исторических выплат ушли провайдеру, который в тот момент по правилам допуска вообще не мог их принять. Это характеристика данных — и лучший аргумент за то, что слой hard-constraints нужен отдельно.

**26 п.п. против 38 п.п.**

balanced против conversion\_first на тех же 100 операциях. Взамен конверсия 0.66 против 0.69.

Три пункта конверсии стоят двенадцати пунктов отклонения от целевых долей — решение показывает цену, а не выбирает за бизнес.

# Что нужно изменить — и сколько для этого строк кода

| Задача                                          | Что делаем                                               | Строк кода |
|-------------------------------------------------|----------------------------------------------------------|------------|
| Добавить провайдера                             | Строка в providers.json                                  | 0          |
| Изменить лимиты, банки, конверсию, целевые доли | Правка providers.json                                    | 0          |
| Изменить приоритеты бизнеса                     | Веса профиля в config/routing.yml                        | 0          |
| Изменить порядок разрешения конфликтов          | Ключ conflict_order в конфиге                            | 0          |
| Выключить любое правило допуска                 | Ключ в разделе constraints конфига                       | 0          |
| Другая политика роутинга целиком                | Новый профиль + флаг --profile                           | 0          |
| Новое правило допуска                           | Класс в constraints/ + строка в реестре + ключ в конфиге | ~25        |
| Новая стратегия                                 | Класс в strategies/ + строка в реестре + вес в профиле   | ~30        |

Ни одно имя провайдера не зашито в код. Семь профилей уже в репозитории: **balanced**, cascade, traffic, volume, conversion\_first, commitments, demo\_failover. Сравнить их на одних данных — **rake compare**.

# Что сделано и как это проверить

|                      |                     |                      |                   |                        |
|----------------------|---------------------|----------------------|-------------------|------------------------|
| 116                  | 29 / 29             | 0                    | 7                 | CI                     |
| тестов, 776 проверок | проверок валидатора | внешних зависимостей | профилей роутинга | зелёный на каждый push |

| Проверить самим                           | Команда                                     |
|-------------------------------------------|---------------------------------------------|
| Тесты без единой внешней зависимости      | rake test                                   |
| Прогон публичной очереди из 10 заявок     | rake route                                  |
| Валидатор организаторов поверх результата | rake validate                               |
| Каскад и fallback в действии              | rake demo                                   |
| Разбор истории и сравнение политик        | rake analyze · rake backtest · rake compare |
| Сдаваемые файлы из тестовой очереди       | rake submit                                 |

Ограничения кейса соблюдены: нейросетей внутри решения нет, проприетарных компонентов нет, весь код — Ruby на стандартной библиотеке.

## Почему веса именно такие

- Веса — не подобранная магия, а расстановка приоритетов, которую можно защитить словами. **Доли по количеству и объёму вместе дают 0.42** — это прямое требование ТЗ и договора.
- **Конверсия 0.18** — второй по величине одиночный вес: отправлять туда, где выплата не проходит, дороже, чем немного отклониться от доли.
- **Фин. обязательства 0.14** — вес умеренный, потому что стратегия и так разрывная: она выстреливает только когда минимум не набран.
- **Каскад, диапазон суммы, загрузка — 0.26 на троих**: это уточняющие сигналы. Жёсткие версии этих же ограничений уже отработали в слое допуска.
- Проверка на истории: профили traffic, volume, commitments и balanced дают одинаковые 26 п.п. Значит, результат устойчив к перекосу весов, а не держится на одной подобранной комбинации.

Веса меняются в config/routing.yml без правки кода — и rake compare сразу показывает цену изменения.

## Нормировка: почему часть сигналов сравнительная, а часть абсолютная

### Сравнительные — min-max по пулу

traffic\_share, volume\_share, conversion, cascade\_priority.

Разброс конверсий в кейсе — 0.79...0.91. Без нормировки эта разница растворяется среди остальных слагаемых и не влияет ни на что.

### Абсолютные — по своей шкале

load\_balance, turnover\_commitment, amount\_band.

«Загружен на 64 %» осмысленно само по себе, сравнивать не с чем и не нужно.

**Следствие, о котором стоит знать честно.** При двух кандидатах сравнительные стратегии дают ровно 1.0 и 0.0, и выбор превращается во взвешенное голосование. На публичной очереди из 10 заявок это видно: 4 заявки безальтернативны, на остальных шести все содержательные профили сходятся к одному решению. Расхождение политик проявляется на объёме — поэтому у нас есть backtest.



# Обработка ошибок и граничных ситуаций

| Ситуация                                            | Поведение                                                   |
|-----------------------------------------------------|-------------------------------------------------------------|
| Файл не найден, битый JSON                          | Ошибка с путём и причиной разбора, а не стектрейс           |
| amount строкой, нулём, отрицательным; заявка без id | Отклоняется на загрузке с указанием поля и значения         |
| Дубли operation_id                                  | Отклоняются: молча смаршрутизировать дубль хуже, чем упасть |
| Очередь — один объект вместо массива                | Принимается: валидатор организаторов допускает оба варианта |
| Пустая очередь                                      | Валидный отчёт с нулями, без деления на ноль                |
| providers.json не массив / без обязательного поля   | Ошибка с указанием, чего именно не хватает                  |
| Неизвестная стратегия или правило в конфиге         | Ошибка при старте, а не тихое игнорирование                 |
| Под заявку не подходит вообще никто                 | Уходит на spacerepayments, помечается fallback_used         |
| daily_amount_limit = null                           | Отсутствие лимита, а не ноль                                |
| У провайдера кончились реквизиты посреди очереди    | Покидает пул, остальные заявки идут дальше                  |

Всё перечисленное закрыто тестами: 11 в loaders\_test.rb, 9 в config\_test.rb, 16 в report\_test.rb.

## Открытые вопросы к жюри — и что переключается одним флагом

| Вопрос                                                                            | Наша трактовка                                                                 | Если ответ другой                                                                                                                             |
|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Неуспешная выплата — это отказ, требующий перемаршрутизации, или финал?           | Финал. К следующему переходим, только если провайдер не принял заявку в работу | Флаг <code>retry_on_terminal_failure</code> в конфиге, затем <code>rake submit</code>                                                         |
| Скрытая проверка считает лимиты по исходному снимку <code>providers.json</code> ? | Да — поэтому мы намеренно строже: резервируем лимит в момент выбора            | Ослабить резервирование в конфиге                                                                                                             |
| Можно ли добавлять свои поля в решение?                                           | Да, ТЗ прямо разрешает дополнительные поля                                     | Убрать <code>score</code> , <code>score_breakdown</code> , <code>stage</code> , <code>eligible_providers</code> из <code>Decision#to_h</code> |
| Дополнительные поля провайдера — в конфиг или в <code>providers.json</code> ?     | Держим в конфиге, но код читает оба источника                                  | Ничего менять не нужно                                                                                                                        |

По каждой спорной трактовке мы выбрали более строгий вариант и заранее подготовили переключатель. Если ответ жюри окажется другим — меняется конфиг, а не код, и файлы регенерируются одной командой.